

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion

Carrera de Ingenieria de Software

Aplicaciones Distribuidas (ISR-701) – Unidad 3: Bases de Datos Distribuidas

# Analisis Critico del Modelo de Consistencia y Replicacion en Bases de Datos Distribuidas

Caso de estudio: Sistema de Control de Laboratorios Informaticos (SCLI)

**Codigo: TA-IND-02**

Trabajo Autonomo Individual – Sumativa

**Estudiante:** Isaias Abraham Urbina Romero  
**Equipo PFC:** FUVV – Sistema de Control de Laboratorios Informaticos (SCLI)  
**Asignatura:** Aplicaciones Distribuidas (ISR-701)  
**Unidad:** 3 – Bases de Datos Distribuidas  
**Docente:** Prof. Guerrero Ulloa G.C.  
**Periodo:** 2026–2027  
**Fecha:** 14/07/2026

**Repositorio del PFC:**

[https://github.com/IsaiasUrb/  
TA-IND-02-Analisis-de-Consistencia-y-Replicacion-en-BDD-Distribuidas.git](https://github.com/IsaiasUrb/TA-IND-02-Analisis-de-Consistencia-y-Replicacion-en-BDD-Distribuidas.git)

## Resumen

Este informe analiza el modelo de consistencia y la estrategia de replicacion implementados en SCLI, el sistema distribuido de gestion de laboratorios informaticos desarrollado como Proyecto Fin de Curso del equipo FUVV. SCLI se organiza como un conjunto de microservicios con una base de datos independiente por servicio, sobre el motor distribuido CockroachDB (y PostgreSQL en el servicio aun en construccion), coordinados mediante llamadas REST sincronas y control de concurrencia optimista basado en versionado. Se caracteriza el modelo realmente implementado – consistencia fuerte dentro del limite transaccional de cada microservicio y consistencia debil, de tipo eventual acotada por contrato, entre servicios – y se contrasta con dos alternativas: una arquitectura de sagas con mensajeria asincrona y un despliegue realmente distribuido y multinodo de CockroachDB. El analisis se apoya en los marcos CAP y PACELC para argumentar que la decision de priorizar consistencia sobre disponibilidad dentro de cada agregado (solicitud-reserva) es adecuada para un dominio donde una doble reserva del mismo laboratorio es inaceptable, mientras que la ausencia de compensaciones formales entre servicios y el despliegue actual en nodo unico constituyen limitaciones que deben resolverse antes de produccion. Se concluye que el modelo actual es parcialmente optimo: correcto en su nucleo transaccional, pero incompleto en su frontera distribuida.

## Contents

|                                                                    |          |
|--------------------------------------------------------------------|----------|
| <b>Resumen</b>                                                     | <b>1</b> |
| <b>1 Introduccion</b>                                              | <b>3</b> |
| <b>2 Marco teorico</b>                                             | <b>3</b> |
| 2.1 El espectro de modelos de consistencia . . . . .               | 3        |
| 2.2 Teoremas de compromiso: CAP y PACELC . . . . .                 | 4        |
| 2.3 Estrategias de replicación . . . . .                           | 4        |
| <b>3 Analisis del modelo de consistencia del PFC</b>               | <b>5</b> |
| 3.1 Arquitectura y topologia de datos de SCLI . . . . .            | 5        |
| 3.2 Consistencia dentro del limite de cada microservicio . . . . . | 5        |
| 3.3 Consistencia entre microservicios . . . . .                    | 6        |
| <b>4 Evaluacion de alternativas y trade-offs</b>                   | <b>7</b> |
| <b>5 Discusion critica: ¿es el modelo optimo?</b>                  | <b>7</b> |
| <b>6 Conclusiones</b>                                              | <b>9</b> |
| <b>7 Declaracion de uso de IA generativa</b>                       | <b>9</b> |

## 1. Introduccion

El Proyecto Fin de Curso (PFC) del equipo FUVV, denominado SCLI (Sistema de Control de Laboratorios Informáticos), tiene como dominio de negocio la gestión distribuida de laboratorios de cómputo de una institución universitaria: registro académico de laboratorios, equipos y horarios; autenticación y autorización de usuarios; y el ciclo completo de solicitudes, aprobaciones y reservas de uso de laboratorios. El sistema reemplaza una aplicación monolítica previa por una arquitectura de microservicios, decisión que traslada al equipo de desarrollo la responsabilidad de definir explícitamente cómo se replican los datos y qué garantías de consistencia ofrece el sistema ante escrituras concurrentes.

Esta responsabilidad es especialmente crítica en el dominio de SCLI porque el recurso que se protege – la franja horaria de un laboratorio – es físicamente no compartible: dos solicitudes aprobadas para el mismo laboratorio, fecha y horario representan un error de negocio, no una inconsistencia tolerable. Al mismo tiempo, el sistema debe seguir respondiendo con baja latencia a consultas de disponibilidad y a la mayoría de operaciones administrativas, que no comparten esa misma exigencia.

El objetivo de este informe es doble. Primero, fundamentar teóricamente el espectro de modelos de consistencia, los teoremas CAP y PACELC, y las estrategias de replicación relevantes para sistemas de bases de datos distribuidas (Sección 4). Segundo, aplicar ese marco al análisis crítico del modelo realmente implementado en SCLI (Sección 5), evaluarlo frente a alternativas concretas (Sección 6) y responder, con argumentos técnicos y no con preferencias personales, la pregunta orientadora del trabajo: *¿es el modelo de consistencia elegido el óptimo para el dominio del sistema?* (Sección 7).

Es pertinente aclarar que SCLI se encuentra en desarrollo activo: de los cuatro microservicios que lo componen, tres están contenerizados y operativos (`auth-service`, `usuarios-service`, `academico-laboratorios-service`) y el cuarto, `reservas-solicitudes-service` – que contiene precisamente la lógica de consistencia más exigente del sistema – está implementado a nivel de dominio, máquina de estados y persistencia, pero aún no se ha incorporado al despliegue orquestado. El análisis que sigue se basa en el código fuente, los esquemas de base de datos, la configuración de despliegue y los documentos de diseño de dominio efectivamente presentes en el repositorio del equipo (<https://github.com/IsaiasUrb/TA-IND-02-Analisis-de-Consistencia-y-Replicacion> `git`), y distingue explícitamente entre lo implementado y lo pendiente.

## 2. Marco teorico

### 2.1. El espectro de modelos de consistencia

La consistencia de réplicas no debe entenderse como una propiedad binaria (consistente/inconsistente), sino como un espectro de garantías con semántica formal precisa, ordenadas por la fuerza de la garantía que ofrecen al cliente [6]. En el extremo más fuerte, la *linealizabilidad* (o consistencia estricta) exige que toda operación parezca ejecutarse de forma instantánea en algún punto entre su invocación y su respuesta, de modo que exista un orden global único compatible con el tiempo real observado por los clientes [6]. La *consistencia secuencial* relaja la coincidencia con el tiempo real pero conserva un orden total único, igual para todos los procesos, en el que las operaciones de cada proceso individual respetan su orden de programa [5].

Por debajo de estos modelos globales aparecen las garantías centradas en el cliente. La *consistencia causal* solo exige que las operaciones causalmente relacionadas – por ejemplo, una lectura que sigue a una escritura del mismo valor – se observen en el mismo orden por todos los procesos, permitiendo que operaciones concurrentes e independientes se observen en órdenes distintos

[6]. La *consistencia de lectura de los propios cambios* (read-your-writes) garantiza únicamente que un proceso siempre observe sus propias escrituras previas, sin comprometerse respecto a las escrituras de otros procesos [6]. Finalmente, la *consistencia eventual* es la garantía más débil de las consideradas: si no se producen nuevas escrituras, todas las réplicas convergen eventualmente al mismo estado, pero durante la propagación pueden observarse valores obsoletos sin ningún límite temporal explícito [6].

La relación entre estos modelos y el costo de coordinación es directa: un modelo más fuerte exige que las réplicas se sincronicen – mediante consenso, bloqueo distribuido o un único nodo autoritativo – antes de confirmar una operación, lo que incrementa la latencia y reduce la disponibilidad ante fallos o particiones de red. Un modelo más débil permite que cada réplica responda con su estado local sin coordinar con las demás, lo que maximiza disponibilidad y minimiza latencia a costa de admitir divergencia temporal entre réplicas [5].

## 2.2. Teoremas de compromiso: CAP y PACELC

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura de Brewer, establece que un sistema distribuido que replica datos no puede garantizar simultáneamente consistencia (C) y disponibilidad (A) durante una partición de red (P): ante la partición, el sistema debe elegir entre rechazar operaciones para preservar la consistencia, o responder con datos potencialmente desactualizados para preservar la disponibilidad [4].

CAP describe únicamente el comportamiento del sistema durante una partición, lo cual resulta insuficiente para caracterizar sistemas que rara vez sufren particiones pero que enfrentan permanentemente el compromiso entre consistencia y latencia en operación normal. La formulación PACELC, propuesta por Abadi, extiende el análisis: si hay Partición (P), el sistema elige entre Disponibilidad (A) y Consistencia (C), igual que en CAP; pero Si no (Else), en operación normal, el sistema elige entre Latencia (L) y Consistencia (C) [1]. Esta segunda rama es la que explica, por ejemplo, por qué un sistema puede optar por replicación sincrónica – mayor consistencia, mayor latencia – incluso sin que exista ninguna partición activa. Leer la decisión de diseño de un sistema exclusivamente bajo CAP, ignorando el eje EL de PACELC, produce un análisis incompleto, motivo por el cual este informe utiliza ambos marcos de forma conjunta en la Sección 5.

## 2.3. Estrategias de replicación

La estrategia de replicación determina cómo se propagan las escrituras entre nodos y condiciona directamente el modelo de consistencia alcanzable. En la *replicación primario-respaldo* (leader-follower), todas las escrituras se dirigen a un único nodo líder que las ordena y propaga a los seguidores; es sencilla de razonar y evita conflictos de escritura, pero el líder es un punto único de coordinación y, en su variante sincrónica, también un limitante de disponibilidad [5]. En la *replicación multi-líder*, varios nodos aceptan escrituras de forma independiente y luego reconcilian sus estados, lo que mejora la disponibilidad y la latencia local a costa de introducir conflictos de escritura que deben resolverse – típicamente mediante marcas de tiempo, vectores de versión o lógica de negocio [5]. En la *replicación sin líder por quorum*, cualquier réplica puede aceptar lecturas y escrituras, y la consistencia se controla mediante los parámetros de quorum de lectura y escritura ( $R + W > N$ ), de modo que el sistema puede ajustar el compromiso entre consistencia y disponibilidad sin cambiar de arquitectura [5].

Sobre este eje se distingue además la *replicación sincrónica*, en la que una escritura no se confirma al cliente hasta que un número suficiente de réplicas la ha aplicado – maximizando consistencia y durabilidad a costa de latencia y disponibilidad – frente a la *replicación asíncrona*, en la que el nodo primario confirma la escritura antes de que las réplicas la hayan aplicado, minimizando

la latencia percibida a costa de una ventana de inconsistencia y de posible pérdida de datos si el primario falla antes de propagar [5].

Los sistemas de referencia industrial ilustran los extremos de este espectro. Dynamo, el almacén clave-valor de Amazon, prioriza la disponibilidad mediante una arquitectura sin líder, particionada por hashing consistente y replicada por quorum configurable, ofreciendo consistencia eventual y resolución de conflictos en el lado de lectura [3]. En el extremo opuesto, Spanner, la base de datos globalmente distribuida de Google, ofrece consistencia externa – transacciones globalmente ordenadas de forma equivalente a un sistema no replicado – combinando el protocolo de consenso Paxos para la replicación sincrónica de cada fragmento de datos con TrueTime, un servicio de sincronización temporal con límites de incertidumbre explícitos, para ordenar transacciones entre fragmentos sin necesidad de comunicación adicional [2]. Ambos sistemas son referencias directas para el análisis de SCLI: como se desarrolla en la Sección 5, el motor de base de datos elegido por el equipo FUVV desciende conceptualmente de la familia Spanner.

### 3. Análisis del modelo de consistencia del PFC

#### 3.1. Arquitectura y topología de datos de SCLI

SCLI implementa el patrón de *base de datos por servicio* (database-per-service): cada uno de los cuatro microservicios – `auth-service`, `usuarios-service`, `academico-laboratorios-service` y `reservas-solicitudes-service` – posee su propio esquema y su propia instancia de base de datos, sin claves foráneas ni transacciones compartidas entre límites de servicio. Los tres primeros se despliegan, según el archivo `docker-compose.yml` del repositorio, sobre instancias independientes de CockroachDB en modo `start-single-node`; el cuarto, aun no incorporado al despliegue orquestado, está configurado para PostgreSQL con migraciones versionadas mediante Flyway. La Figura 1 ilustra esta topología.

CockroachDB es, en sí mismo, un motor de base de datos SQL distribuido inspirado directamente en la arquitectura de Spanner: particiona los datos en *ranges*, replica cada *range* mediante el protocolo de consenso Raft, y ofrece por defecto aislamiento serializable en todas las transacciones. La elección de este motor es una decisión de consistencia relevante incluso antes de examinar el código de aplicación, porque establece como piso de garantía la consistencia fuerte dentro de cada base de datos individual. Sin embargo, el despliegue actual con la bandera `start-single-node` implica que, en el estado presente del proyecto, cada instancia de CockroachDB opera como nodo único: la capacidad de replicación y tolerancia a fallos del motor no está siendo explotada todavía, aunque el modelo de consistencia serializable sí se cumple localmente. Esta distinción – entre el modelo de consistencia que el motor puede ofrecer y el que efectivamente ofrece el despliegue vigente – es central para responder la pregunta orientadora en la Sección 7.

#### 3.2. Consistencia dentro del límite de cada microservicio

Dentro del límite transaccional de `reservas-solicitudes-service` – el componente donde reside la lógica de negocio más sensible a condiciones de carrera – el modelo implementado corresponde a consistencia fuerte de tipo serializable, reforzada por dos mecanismos complementarios observables directamente en el código.

Primero, control de concurrencia optimista mediante versionado: las entidades `SolicitudReserva`, `Reserva`, `BloqueoAgenda` y `ConfiguracionReserva` incorporan un campo `version` anotado con `@Version` de JPA/Hibernate. Según el documento de diseño de la máquina de estados del propio equipo, la cancelación concurrente de una reserva “se controla con el campo `version` mediante *versionado optimista*”, de modo que si dos solicitudes de cancelación compiten por la misma

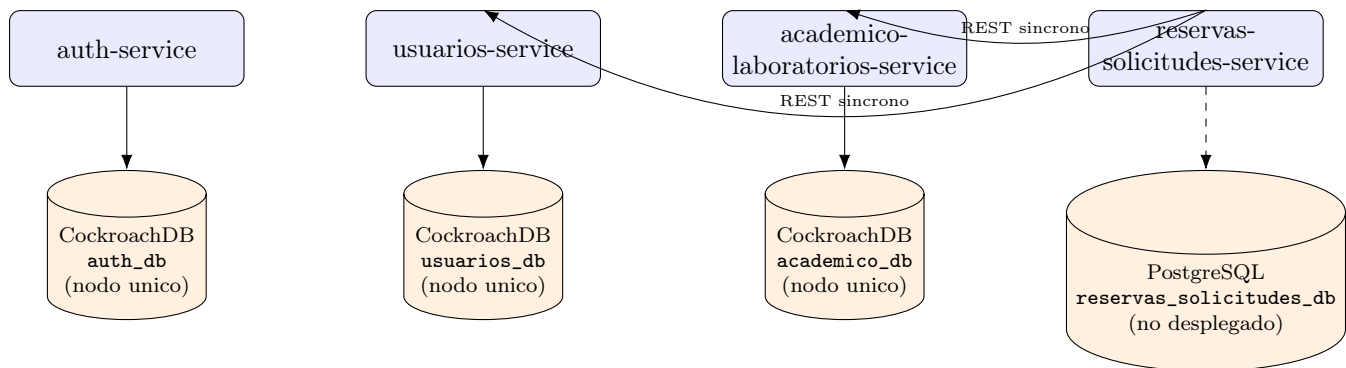


Figure 1: Topología de datos de SCLI: patron de base de datos por servicio. Cada microservicio posee un limite transaccional propio y una instancia de base de datos exclusiva; **reservas-solicitudes-service** consulta a **academico-laboratorios-service** y a **usuarios-service** mediante llamadas REST sincronas sin transaccion distribuida. El cuarto servicio (línea discontinua) aun no forma parte del despliegue orquestado con **docker-compose**.

reserva, la que pierde la carrera no puede sobrescribir el estado vigente y recibe una respuesta de conflicto. Este mecanismo evita, sin necesidad de bloqueos pesimistas permanentes, que dos actores modifiquen concurrentemente el mismo agregado sin darse cuenta uno del otro.

Segundo, atomicidad de agregado mediante transaccion local unica: la aprobacion de una solicitud – que cambia su estado a **APROBADA**, crea exactamente una **Reserva** en estado **PROGRAMADA** y registra el evento en **HistorialSolicitud** – esta explicitamente definida como una unica operacion de negocio que debe ejecutarse en la misma transaccion local, revirtiendose por completo si cualquiera de sus efectos falla. La misma exigencia se aplica a la cancelacion de una solicitud aprobada, que debe cambiar de estado tanto la solicitud como su reserva asociada dentro de una unica transaccion. Este diseño evita, por construccion, estados intermedios inconsistentes como una solicitud aprobada sin reserva asociada o una reserva huérfana sin solicitud.

A esto se suma el uso de una cabecera **Idempotency-Key**, obligatoria en las operaciones de creacion y aprobacion de solicitudes, que protege al sistema frente a reintentos de red: una misma clave de idempotencia repetida no debe producir efectos duplicados, propiedad indispensable en un sistema distribuido donde el cliente no siempre puede saber con certeza si una peticion previa tuvo exito en el servidor.

En conjunto, dentro de cada microservicio SCLI implementa un modelo cercano a la linealizabilidad para las operaciones que modifican el agregado **SolicitudReserva/Reserva**: cualquier lector posterior a una escritura confirmada observa el efecto de esa escritura, porque no existe replica secundaria dentro del mismo servicio que pueda quedarse atras (el despliegue es de nodo unico) y porque el aislamiento serializable de CockroachDB, combinado con el versionado optimista de la aplicacion, impide anomalías de escritura concurrente.

### 3.3. Consistencia entre microservicios

El panorama cambia en la frontera entre servicios. Cuando **reservas-solicitudes-service** necesita verificar la existencia de un laboratorio o la disponibilidad de un docente, invoca sincronamente a **academico-laboratorios-service** y a **usuarios-service** mediante clientes REST configurados con tiempos de espera y reintentos (**RestClientConfig**, **AcademicoLaboratoriosClient**, **UsuariosClient**). No existe entre estos servicios ninguna transaccion distribuida, protocolo de confirmacion en dos fases ni mensajería asincrona: el repositorio no contiene dependencias de Kafka, RabbitMQ ni ningun otro middleware de mensajería. Los propios documentos de diseño del equipo son explicitos al respecto: las referencias externas como **laboratorioId** o



`solicitanteId` se almacenan como valores UUID *"sin claves foráneas hacia bases de datos externas y sin relaciones ORM locales"*, y su validez *"se resolverá mediante los contratos de integración correspondientes, no mediante integridad referencial entre bases"*.

Esta decisión de diseño equivale, en términos del espectro de la Sección 4.1, a un modelo de consistencia débil entre servicios – más cercano a la consistencia eventual acotada por contrato que a cualquier garantía fuerte – con una ventana de posible obsolescencia entre el instante en que `reservas-solicitudes-service` consulta el estado de un laboratorio (`DisponibilidadController`) y el instante en que aprueba una solicitud sobre ese mismo laboratorio. Si `academico-laboratorios-service` cambia el estado de un laboratorio a inactivo entre ambas llamadas, no existe ningún mecanismo de coordinación distribuida que lo impida a nivel de infraestructura; la coherencia depende de que la aplicación vuelva a validar la condición en el momento de aprobar, no de una garantía transaccional entre bases de datos. Es, en esencia, el mismo compromiso que describe la formulación PACELC para el eje Else-Latencia-Consistencia: el equipo optó por baja latencia y desacoplamiento entre servicios en operación normal, aceptando una consistencia más débil en las lecturas entre límites de servicio.

## 4. Evaluación de alternativas y trade-offs

Para responder si el modelo vigente es el óptimo, se contrastan a continuación dos alternativas concretas frente al modelo actualmente implementado.

**Alternativa 1 – Sagas coreografiadas con mensajería asíncrona.** En lugar de que `reservas-solicitudes-service` invoque sincronamente a los demás servicios y valide en el mismo hilo de petición, el flujo de aprobación se modelaría como una saga: cada paso (reservar horario, validar disponibilidad de laboratorio, notificar) se publica como evento en un broker (por ejemplo Kafka o RabbitMQ) y cada servicio reacciona de forma asíncrona, con una acción de compensación explícita si un paso posterior falla (por ejemplo, liberar la reserva si la validación académica llega negativa). Este patrón es el que sistemas orientados a disponibilidad, como Dynamo, generalizan mediante resolución de conflictos y reconciliación posterior en lugar de coordinación previa [3].

**Alternativa 2 – Clúster CockroachDB realmente distribuido (multinodo).** Sin cambiar el modelo de programación actual, se sustituirían las instancias `start-single-node` por un despliegue de al menos tres nodos por servicio, habilitando la replicación Raft real entre ellos. El modelo de consistencia dentro de cada servicio no cambiaría – seguiría siendo serializable – pero pasaría de ser una propiedad teórica del motor a una garantía efectiva con tolerancia a fallos de nodo, acercando el comportamiento del sistema al de Spanner, del cual CockroachDB descende conceptualmente [2].

La Tabla 1 contrasta ambas alternativas con el modelo actual según cinco criterios técnicos.

Las dos alternativas no son mutuamente excluyentes ni compiten por el mismo problema: la Alternativa 2 resuelve la fragilidad de la replicación *dentro* de cada servicio, mientras que la Alternativa 1 resuelve la fragilidad de la coordinación *entre* servicios. Esta distinción es la base de la discusión crítica de la Sección 6.

## 5. Discusión crítica: ¿es el modelo óptimo?

La respuesta a la pregunta orientadora no es uniforme para todo el sistema, porque SCLI en realidad contiene dos modelos de consistencia distintos según el nivel al que se observe.

Dentro del agregado `SolicitudReserva/Reserva`, el modelo elegido – consistencia serializable respaldada por versionado optimista y transacciones locales atómicas – es técnicamente ade-

Table 1: Comparacion del modelo actual de SCLI frente a dos alternativas de consistencia y replicacion.

| Criterio                 | Modelo actual                                                                                  | Alt. 1: Sagas asincronas                                                       | Alt. 2: Cockroach multinodo                                                  |
|--------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Consistencia             | Fuerte intra-servicio (serializable); debil entre servicios (sin compensacion)                 | Eventual explicita entre servicios, con compensacion definida                  | Fuerte intra-servicio, igual que hoy; no mejora la frontera entre servicios  |
| Disponibilidad           | Media: una llamada REST caida bloquea la operacion completa                                    | Alta: los servicios se desacoplan temporalmente entre si                       | Alta dentro de cada servicio: tolera caida de un nodo sin perder el servicio |
| Latencia                 | Baja-media: suma de latencias de llamadas sincronas encadenadas                                | Baja percibida por el cliente, pero mayor tiempo hasta consistencia final      | Baja-media, ligeramente mayor que nodo unico por el consenso Raft real       |
| Tolerancia a particiones | Baja: una particion hacia academico-laboratorio <del>post-servicio</del> detiene la aprobacion | Alta: los eventos se reintentan cuando la <del>post-servicio</del> se resuelve | Alta: el clúster sigue disponible si falla una minoria de nodos              |
| Complejidad              | Baja: llamadas REST directas, facil de razonar y depurar                                       | Alta: exige broker, logica de compensacion y observabilidad distribuida        | Media: solo cambia infraestructura de despliegue, no logica de aplicacion    |

cuado para el dominio. El costo de una anomalia de consistencia en este punto (dos reservas aprobadas para el mismo laboratorio y horario) es alto y dificil de revertir socialmente una vez que los usuarios han sido notificados, mientras que el costo de rechazar ocasionalmente una operacion concurrente con un **409 Conflict** es bajo: el usuario simplemente reintenta. Leido bajo PACELC, el equipo eligio consistencia sobre latencia en el eje Else, lo cual es coherente con un dominio donde las operaciones de escritura sobre una misma reserva son poco frecuentes por recurso individual y donde la correctitud importa mas que los milisegundos de respuesta.

Sin embargo, dos limitaciones concretas impiden calificar el modelo completo como optimo en su estado actual. La primera es el despliegue en nodo unico: el motor CockroachDB fue elegido precisamente por su capacidad de ofrecer consistencia fuerte con replicacion distribuida, pero esa capacidad no esta siendo utilizada. Bajo el eje P de CAP, una particion o caida del unico nodo de `academico_db` no se traduce en un compromiso entre consistencia y disponibilidad, sino en indisponibilidad total del servicio, que es precisamente el resultado que la replicacion deberia evitar. La mitigacion es directa y de bajo costo relativo: la Alternativa 2 no exige reescribir logica de aplicacion, solo cambiar la configuracion de despliegue.

La segunda limitacion es estructural: la ausencia de compensaciones formales entre `reservas-solicitudes-serv` y los servicios de los que depende. El propio diseño documental del equipo reconoce que la validez de las referencias externas se resuelve por contrato, no por integridad referencial, lo cual es una decision razonable en microservicios pero que traslada la responsabilidad de la consistencia a la logica de aplicacion en tiempo de peticion. Mientras el sistema sea de tamaño reducido y los servicios respondan con alta disponibilidad, esta ventana de obsolescencia entre validacion y aprobacion es improbable que se manifieste; a medida que el sistema escale o se agregue mayor concurrencia entre periodos de matricula, esa ventana deja de ser un riesgo teorico. La mitigacion recomendada no es necesariamente adoptar la Alternativa 1 de inmediato – su complejidad

operativa es alta para el tamaño actual del equipo – sino, como paso intermedio, revalidar explícitamente el estado del laboratorio dentro de la misma transacción de aprobación y documentar el comportamiento esperado ante una respuesta tardía o fallida del servicio académico, antes de considerar mensajería asíncrona como evolución futura.

En síntesis, el modelo de consistencia de SCLI es parcialmente óptimo: acertado en la decisión central de proteger el agregado de reserva con consistencia fuerte, e insuficiente en dos aspectos de implementación – replicación física no explotada y coordinación entre servicios no formalizada – que son subsanables sin cambiar la arquitectura de fondo. Ninguna de las dos limitaciones invalida la elección del modelo; ambas señalan una brecha entre el modelo de consistencia que el diseño del equipo declara perseguir y el que el despliegue actual efectivamente garantiza.

## 6. Conclusiones

1. SCLI implementa el patrón de base de datos por servicio sobre un motor – CockroachDB – diseñado para ofrecer consistencia fuerte mediante replicación basada en consenso, en la línea de Spanner, aunque su despliegue actual como nodo único no explota todavía esa capacidad.
2. Dentro del límite transaccional de cada microservicio, en particular en **reservas-solicitudes-service**, el modelo implementado combina aislamiento serializable, control de concurrencia optimista por versión y transacciones locales atómicas, lo que produce garantías cercanas a la linealizabilidad para el agregado solicitud-reserva, adecuadas a un dominio donde la doble reserva de un mismo laboratorio es inaceptable.
3. Entre microservicios, la ausencia de transacciones distribuidas y de mensajería asíncrona con compensación formal produce un modelo de consistencia débil, de tipo eventual acotado por contrato, con una ventana de obsolescencia entre la consulta de disponibilidad y la aprobación de una solicitud.
4. Leído bajo CAP y PACELC, el sistema prioriza consistencia sobre latencia dentro de cada servicio y prioriza desacoplamiento y baja latencia sobre consistencia estricta entre servicios; ambas decisiones son defendibles para el dominio, pero la primera no está completamente respaldada aun por la infraestructura de despliegue.
5. El modelo de consistencia elegido es parcialmente óptimo para el dominio de SCLI: la elección conceptual es correcta, pero su realización está incompleta mientras el proyecto continúa en desarrollo, y las dos mitigaciones identificadas – despliegue multinodo y revalidación transaccional entre servicios – son de complejidad manejable para el estado actual del equipo.

## 7. Declaración de uso de IA generativa

Para la elaboración de este informe se utilizó puntualmente la herramienta Claude (Anthropic), únicamente como apoyo en dos tareas específicas: (i) revisión del código fuente y la configuración de despliegue del repositorio del PFC SCLI (<https://github.com/IsaiasUrb/TA-IND-02-Analisis-de-Consistencia>) – en particular el archivo `docker-compose.yml`, las entidades y controladores de **reservas-solicitudes-service** y los documentos de diseño de dominio del equipo – para verificar con precisión técnica el motor de base de datos, la estrategia de replicación y los mecanismos de concurrencia efectivamente implementados antes de caracterizarlos en la Sección 5; y (ii) contraste de las alternativas de la Sección 6 frente al modelo actual, para asegurar que los criterios de comparación fueran técnicamente consistentes con la literatura citada. La redacción, el análisis crítico, la postura frente

a la pregunta orientadora y las conclusiones del informe son elaboracion propia del estudiante, quien conoce el PFC por participar en su desarrollo y valido cada afirmacion tecnica contra el repositorio fuente antes de la entrega.

## References

- [1] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, 2012. doi: 10.1109/MC.2012.33
- [2] J. C. Corbett *et al.*, “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013, art. no. 8. doi: 10.1145/2491245
- [3] G. DeCandia *et al.*, “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proc. 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP ’07)*, 2007, pp. 205–220. doi: 10.1145/1294261.1294281
- [4] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. doi: 10.1145/564585.564601
- [5] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. distributed-systems.net, 2017, isbn: 978-1543057386.
- [6] P. Viotti and M. Vukolic, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, no. 1, pp. 1–34, 2016, art. no. 19. doi: 10.1145/2926965